

Indexing in DBMS

- Indexing is used to **optimize the performance of a database by minimizing the number of disk accesses** required when a query is processed.
- The **index is a type of data structure**. It is used to locate and access the data in a database table quickly.

Index structure:

Indexes can be created using some database columns.

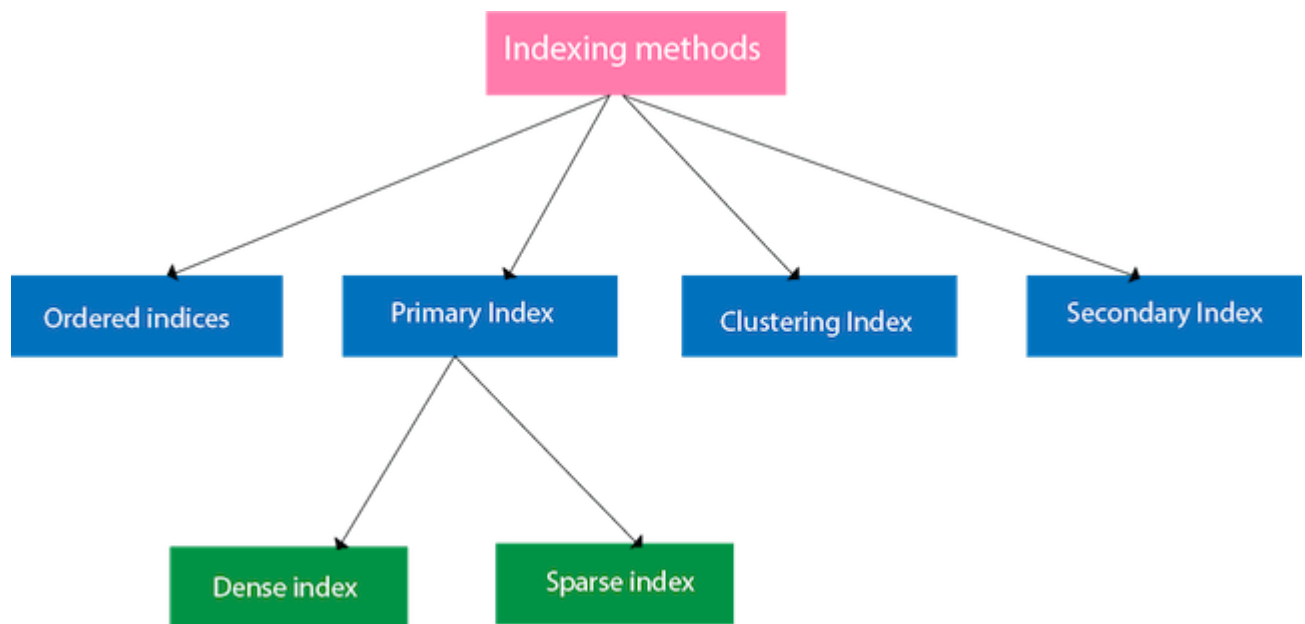
Search key	Data Reference
------------	----------------

Fig: Structure of Index

23089 (value of employee id)	#ffgghh (pointer to the location where this employee id is stored)
------------------------------	--

- The **first column** of the database is the **search key** that contains a copy of the **primary key or candidate key of the table**. The values of the primary key are stored in sorted order so that the corresponding data can be accessed easily.
- The second column of the database is the data reference. It contains a set of **pointers holding the address of the disk block where the value of the particular key can be found**.

Indexing Methods



Ordered indices

The indices are usually sorted to make searching faster. **The indices which are sorted are known as ordered indices.**

Example: Suppose we have an employee table with thousands of record and each of which is 10 bytes long. If their IDs start with 1, 2, 3....and so on and we have to search employee with ID-543.

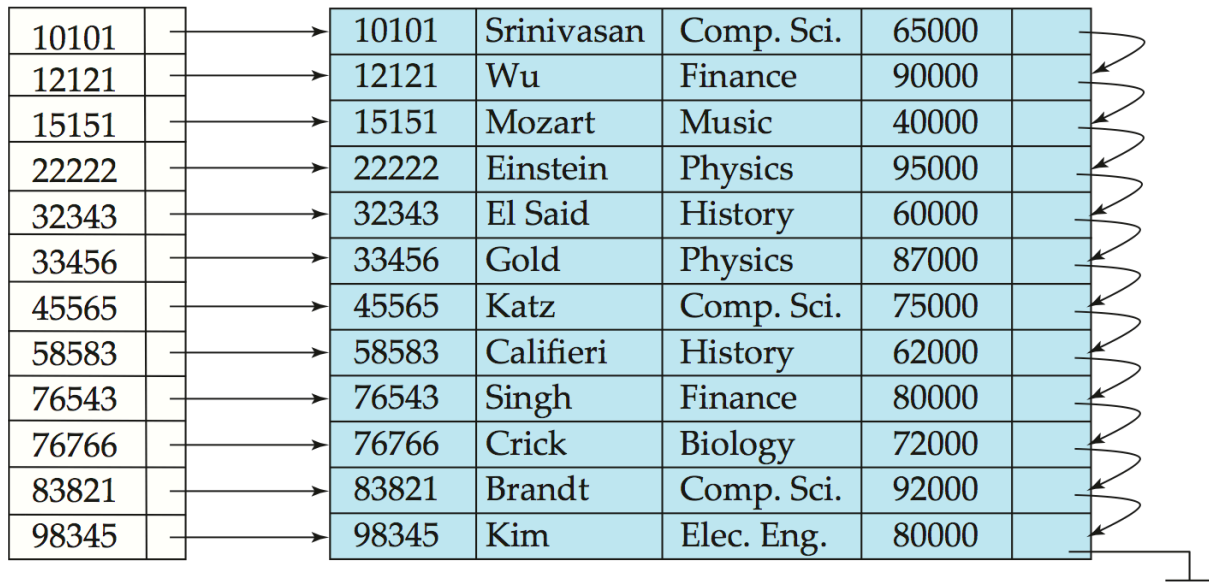
- In the case of a database with no index, we have to search the disk block from starting till it reaches 543. The DBMS will read the record after reading $543 \times 10 = 5430$ bytes.
- In the case of an index, we will search using indexes and the DBMS will read the record after reading $542 \times 2 = 1084$ bytes which are very less compared to the previous case.

Primary Index

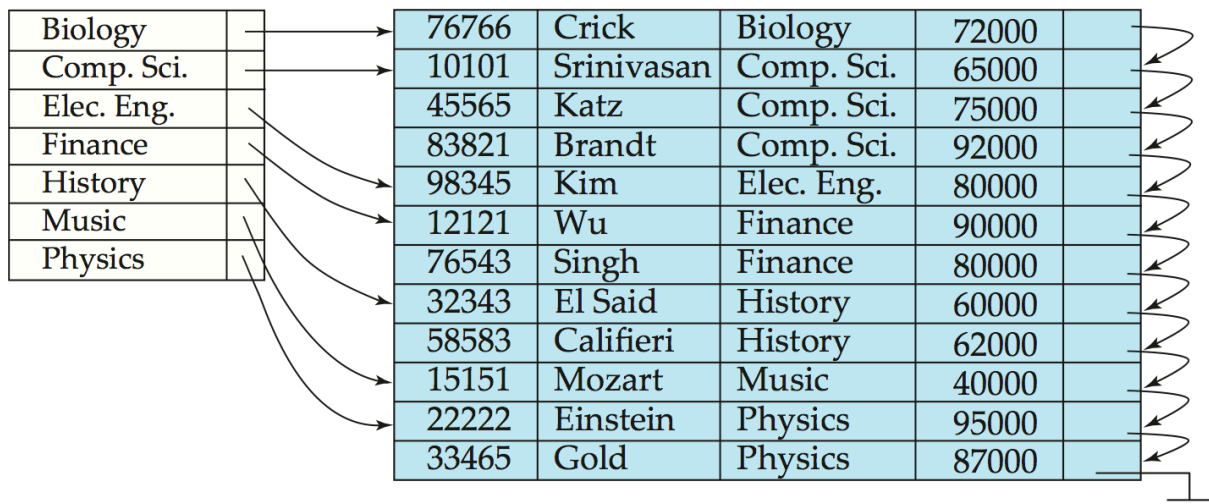
- If the **index is created on the basis of the primary key of the table**, then it is known as primary indexing. These primary keys are unique to each record and contain 1:1 relation between the records.
- As primary keys are stored in sorted order, the performance of the searching operation is quite efficient.
- The primary index can be classified into two types: Dense index and Sparse index.

Dense index

- The **dense index contains an index record for every search key value** in the data file. It makes searching faster.
- In this, the **number of records in the index table is same as the number of records in the main table**.
- It **needs more space to store index record** itself. The index records have the search key and a pointer to the actual record on the disk.

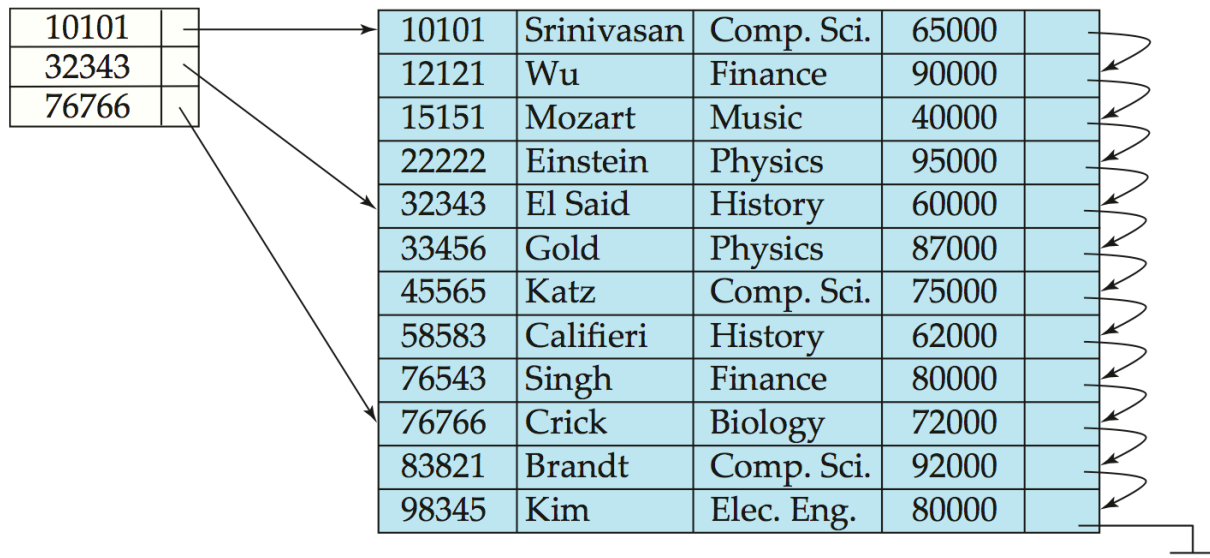


Dense index on non-primary key attribute



Sparse index

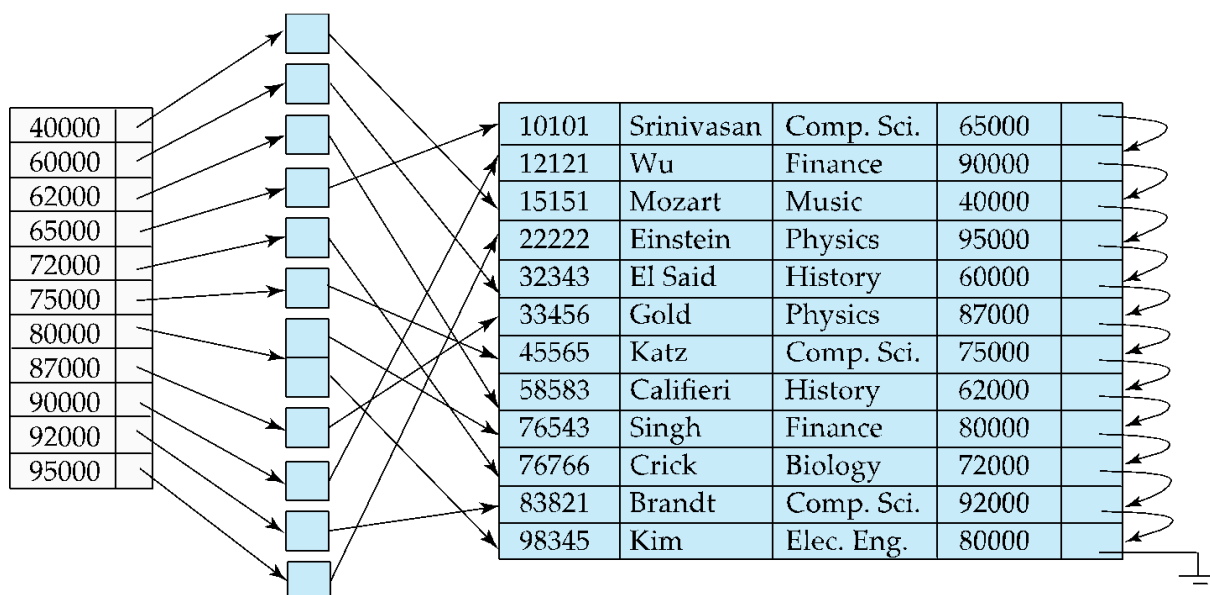
- In the data file, **index record appears only for a few items**. Each item points to a block.
- In this, instead of pointing to each record in the main table, the index points to the records in the main table in a gap.



Secondary index

Index record points to a bucket that contains pointers to all the actual records with that particular search-key value.

Secondary indices have to be dense



Secondary index on salary field of employee

Primary vs secondary indices

Indices offer substantial benefits when searching for records.

BUT: Updating indices imposes overhead on database modification --when a file is modified, every index on the file must be updated,

Sequential scan using primary index is efficient, but a sequential scan using a secondary index is expensive

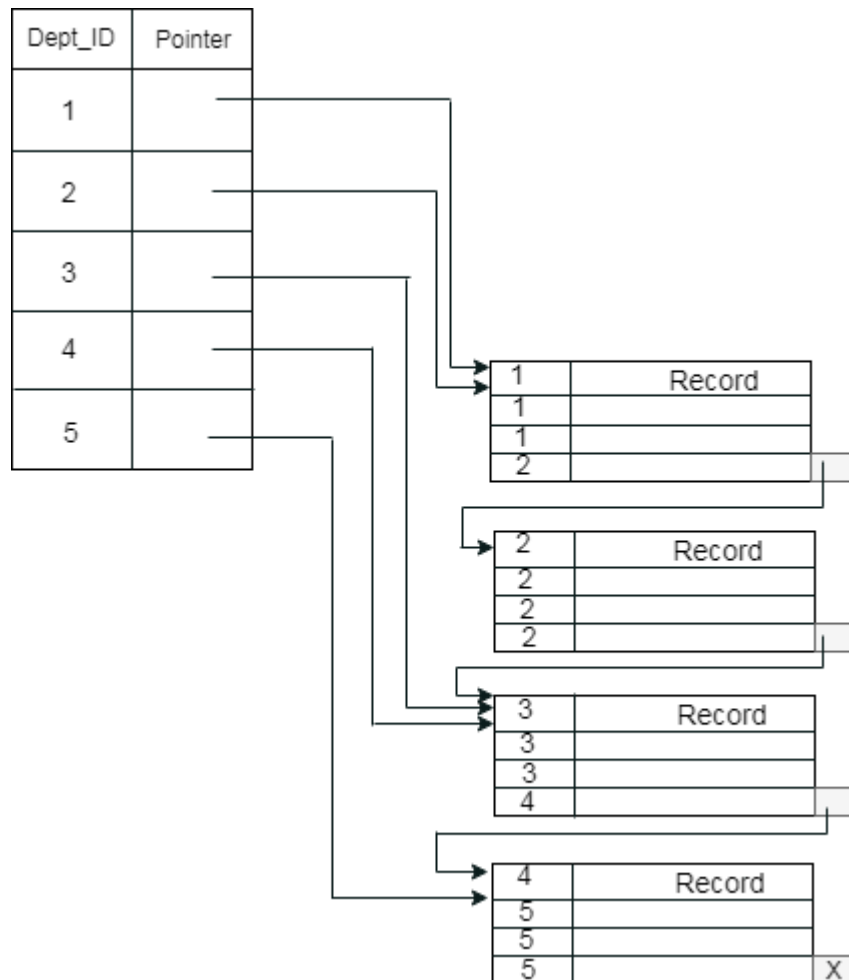
Each record access may fetch a new block from disk

Block fetch requires about 5 to 10 milliseconds, versus about 100 nanoseconds for memory access

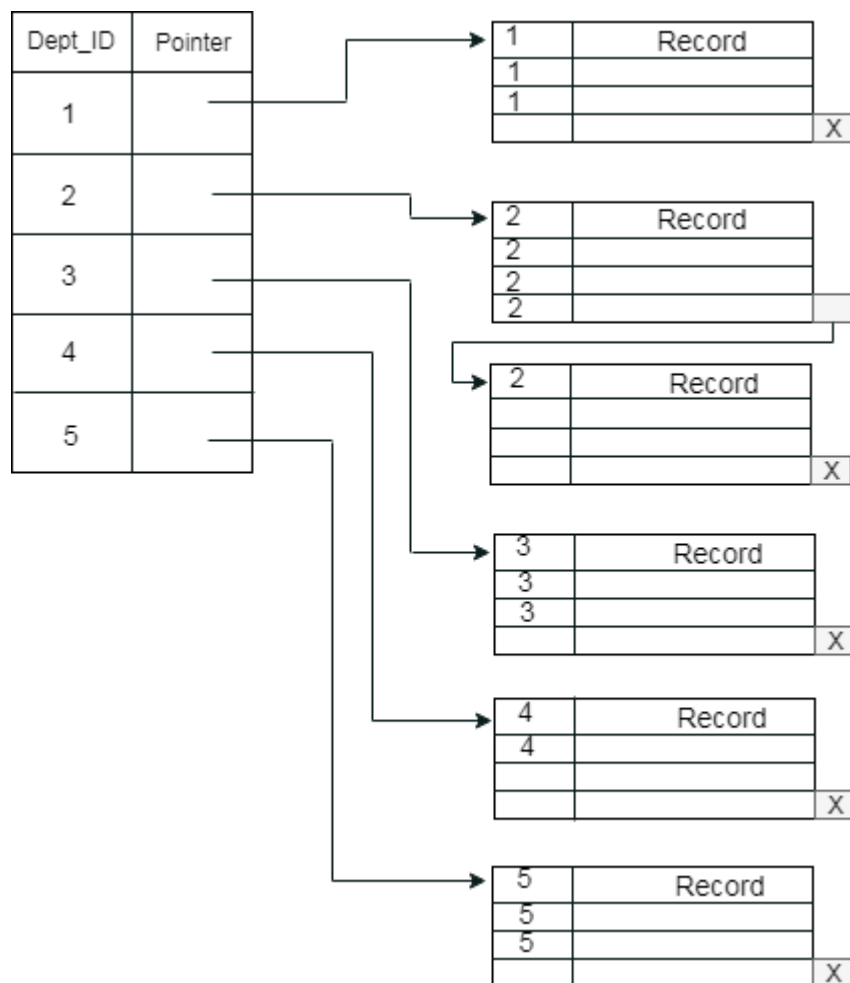
Clustering Index

- A clustered index can be defined as an ordered data file. Sometimes the index is created on non-primary key columns which may not be unique for each record.
- In this case, to identify the record faster, we will group two or more columns to get the unique value and create index out of them. This method is called a clustering index.
- The records which have similar characteristics are grouped, and indexes are created for these group.

Example: suppose a company contains several employees in each department. Suppose we use a clustering index, where all employees which belong to the same Dept_ID are considered within a single cluster, and index pointers point to the cluster as a whole. Here Dept_Id is a non-unique key.



The previous schema is little confusing because one disk block is shared by records which belong to the different cluster. If we use separate disk block for separate clusters, then it is called better technique.



Multilevel Index

If primary index does not fit in memory, access becomes expensive.

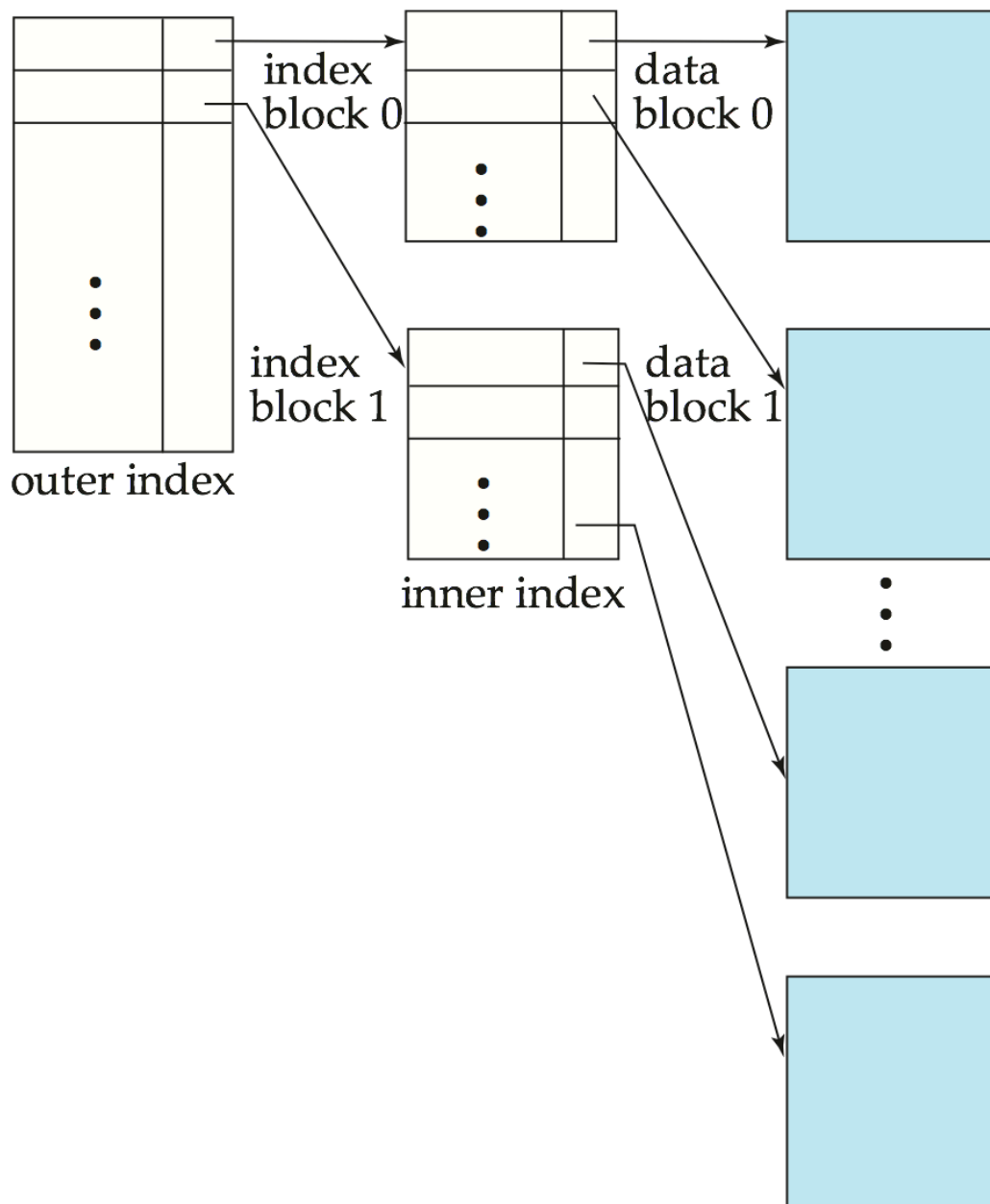
Solution: treat primary index kept on disk as a sequential file and construct a sparse index on it.

outer index – a sparse index of primary index

inner index – the primary index file

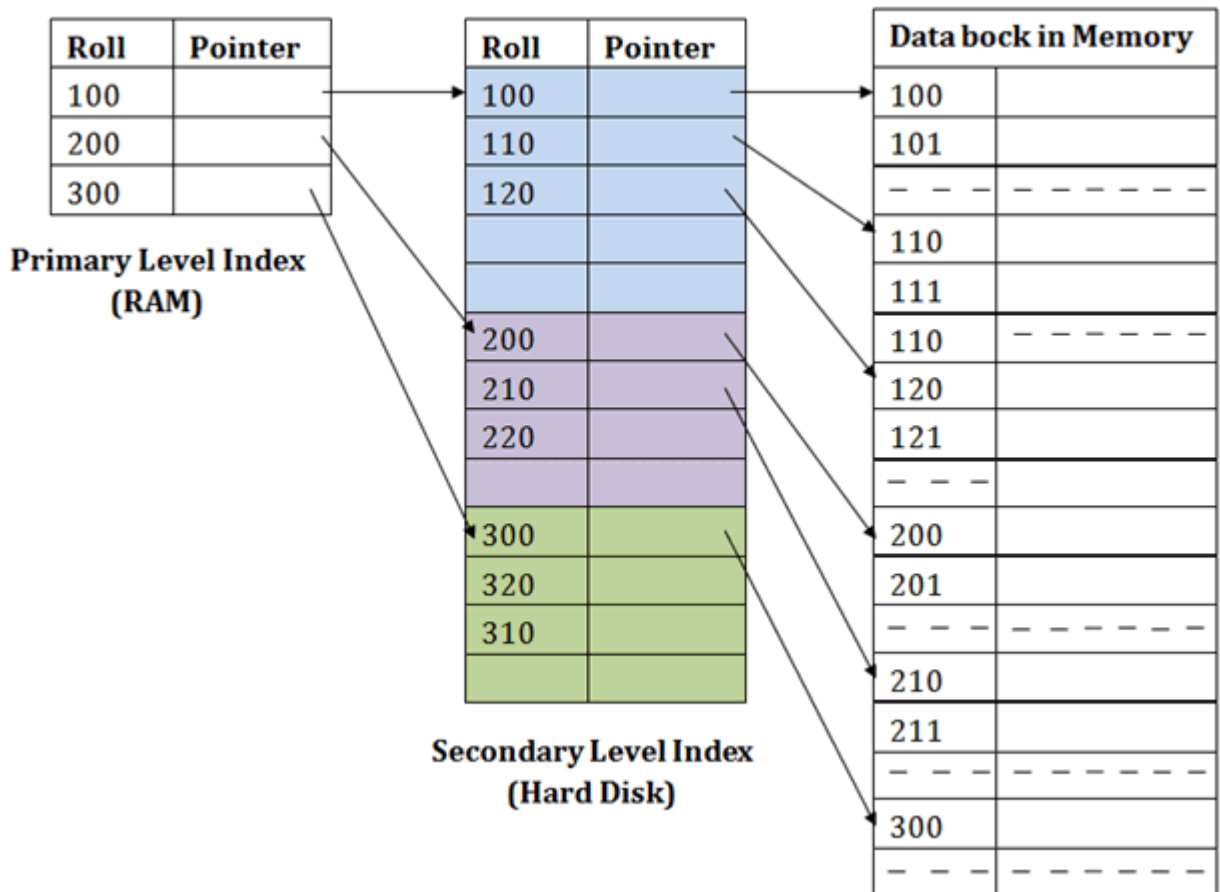
If even outer index is too large to fit in main memory, yet another level of index can be created, and so on.

Indices at all levels must be updated on insertion or deletion from the file.



In the sparse indexing, as the size of the table grows, the size of mapping also grows. These mappings are usually kept in the primary memory so that address fetch should be faster. Then the secondary memory searches the actual data based on the address got from mapping. If the mapping size grows then fetching the address itself becomes slower. In this case, the sparse index will not be efficient. To overcome this problem, multilevel indexing is introduced.

In multilevel indexing, to reduce the size of mapping, another level of indexing is introduced. In this method, the huge range for the columns is selected initially so that the mapping size of the first level becomes small. Then each range is further divided into smaller ranges. The mapping of the first level is stored in the primary memory, so that address fetch is faster. The mapping of the second level and actual data are stored in the secondary memory (hard disk).



For example:

- If you want to find the record of roll 111 in the diagram, then it will search the highest entry which is smaller than or equal to 111 in the first level index. It will get 100 at this level.
- Then in the second index level, again it does $\max(111) \leq 111$ and gets 110. Now using the address 110, it goes to the data block and starts searching each record till it gets 111.
- This is how a search is performed in this method. Inserting, updating or deleting is also done in the same manner.